

Evidence used: README.md, App.tsx, components/, api/, lib/, vite.config.ts, .env.example, supabase/migrations, design/DEFINITIVE-UI-PLAN.md

WHAT IT IS

A mobile-first web app for collecting and enriching geolocated business and infrastructure data from the field. The repo also includes admin review, analytics, and client-facing dashboard flows around that capture pipeline.

WHO IT IS FOR

Primary persona: field agents using phones to capture ground-truth points on the move. Secondary roles are present in code for admins and client/data-consumer users.

HOW TO RUN

- Install deps: `npm install`
- Create local env values from `.env.example` and fill the required secrets, auth, storage, and database settings.
- If you are starting from a fresh database, run `npm run migrate`.
- Start the backend APIs with `npx vercel@latest dev --listen 3000`.
- Start the frontend with `npm run dev`, then open `http://localhost:5173`.

NOT FOUND IN REPO

- Exact local env filename convention (`.env` vs `.env.local`).
- Default demo credentials or a seeded local test account.

WHAT IT DOES

- Map-first exploration and category-based point discovery in the main field UI.
- Multi-step create/enrich submission flow for new points and follow-up edits.
- Live photo upload plus GPS, EXIF, IP, and device-integrity checks on submissions.
- Offline queue with IndexedDB persistence and background sync when connectivity returns.
- XP, streaks, badges, rewards, leaderboard, and level-up feedback for contributors.
- Admin review queue with batch review, fraud-risk grouping, and moderation tools.
- Client/investor dashboards for analytics, deltas, and export-oriented views.

HOW IT WORKS

Compact flow: React app -> lib/client services -> /api endpoints -> storage/blob/AI services -> projected public/admin/client views.

- Frontend: Vite + React SPA. `App.tsx` lazy-loads screen modules such as `Home`, `ContributionFlow`, `AdminQueue`, `DeltaDashboard`, and `InvestorDashboard`, with role-aware navigation.
- Client services: `lib/client` handles auth, API calls, offline queueing, submission sync, motion helpers, GPS/photo integrity checks, and remote wipe behavior.
- Submission path: `ContributionFlow` posts to `/api/submissions`, where validation, rate limiting, dedup checks, fraud scoring, confidence scoring, XP logic, and event projection run server-side.
- Storage layer: `lib/server/storage` resolves Postgres or Edge Config-backed stores, with optional edge fallback during migration; `lib/server/db.ts` uses pg for database access.
- Assets and integrations: submission photos are uploaded to Vercel Blob, auth is handled through `Auth.js` routes, and `/api/ai/search` calls Gemini-backed server search.
- Ops/analytics: `supabase/migrations` defines schema changes, and `api/analytics/index.ts` dispatches scheduled snapshot, trust-decay, and GPS-anomaly jobs.